Name: P.Sanjay Kumar
Roll No: 2403A52L06

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE	DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Program Name: B. Tech	**Assignment Type: Lab**	**Academic Year:**2025-2026
Course Coordinator Name	Dr. Rishabh Mittal	
Instructor(s) Name	Mr. S Naresh Kumar Ms. B. Swathi Dr. Sasanko Shekhar Gantayat Mr. Md Sallauddin Dr. Mathivanan Mr. Y Srikanth Ms. N Shilpa Dr. Rishabh Mittal (Coordinator) Dr. R. Prashant Kumar Mr. Ankushavali MD Mr. B Viswanath Ms. Sujitha Reddy Ms. A. Anitha Ms. M.Madhuri Ms. Katherashala Swetha Ms. Velpula sumalatha Mr. Bingi Raju	

Course Code	23CS002PC304	**Course Title**	AI Assisted Coding
Year/Sem	III/II	**Regulation**	R23
Date and Day of Assignment	**Week3 – Wednesday**	**Time(s)**	23CSBTB01 To 23CSBTB52
Duration	2 Hours	**Applicable to Batches**	All batches

AssignmentNumber:6.3(Present assignment number)/**24**(Total number of assignments)

Q.No.	Question	*Expected Time to complete*
1	**Lab 6: AI-Based Code Completion – Classes, Loops, and Conditionals** **Lab Objectives** • To explore AI-powered auto-completion features for core Python constructs such as classes, loops, and conditional statements. • To analyze how AI tools suggest logic for object-oriented programming and control structures. • To evaluate the correctness, readability, and completeness of AI-generated Python code.	Week3 - Wednesday

Name: P.Sanjay Kumar
Roll No: 2403A52L06

Lab Outcomes (LOs)
After completing this lab, students will be able to:
• Use AI tools to generate and complete Python class definitions and methods.
• Understand and assess AI-suggested loop constructs for iterative tasks.
• Generate and evaluate conditional statements using AI-driven prompts.
• Critically analyze AI-assisted code for correctness, clarity, and efficiency.

Task Description #1: Classes (Student Class)

Scenario
You are developing a simple student information management module.

Task
• Use an AI tool (GitHub Copilot / Cursor AI / Gemini) to complete a Student class.
• The class should include attributes such as name, roll number, and branch.
• Add a method display_details() to print student information.
• Execute the code and verify the output.
• Analyze the code generated by the AI tool for correctness and clarity.

Expected Output #1
• A Python class with a constructor (__init__) and a display_details() method.
• Sample object creation and output displayed on the console.
• Brief analysis of AI-generated code.

Prompt:
Develop a single student information module. Implement a 'student' class with attributes: name, roll number, and branch. Add a 'display_detials()' method to print student information.
Code:

```python
class Student:
    def __init__(self, name, roll_number, branch):
        self.name = name
        self.roll_number = roll_number
        self.branch = branch

    def display_details(self):
        print("Student Details")
        print("----------------")
        print("Name       :", self.name)
        print("Roll No    :", self.roll_number)
        print("Branch     :", self.branch)


# Example usage
student1 = Student("sanjay", "23CS101", "Aiml")
student1.display_details()
```

Output:

Name: P.Sanjay Kumar
Roll No: 2403A52L06

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL

PS C:\Users\sanjay\AI Assistant coding> & C:
oding/lab assignment 6.3.py"
Student Details
----------------
Name        : sanjay
Roll No     : 23CS101
Branch      : Aiml
PS C:\Users\sanjay\AI Assistant coding>
```

Task Description #2: Loops (Multiples of a Number)

Scenario
You are writing a utility function to display multiples of a given number.

Task
• Prompt the AI tool to generate a function that prints the first 10 multiples of a given number using a loop.
• Analyze the generated loop logic.
• Ask the AI to generate the same functionality using another controlled looping structure (e.g., while instead of for).

Expected Output #2
• Correct loop-based Python implementation.
• Output showing the first 10 multiples of a number.
• Comparison and analysis of different looping approaches.
Prompt:
Develop a program that prints first 10 multiples of a given number using a loop
Code:

```python
#task2
num = int(input("Enter a number: "))

print(f"First 10 multiples of {num} are:")

for i in range(1, 11):
    print(num * i)
```

Output:

```
First 10 multiples of 16 are:
16
32
48
64
80
96
112
128
144
160
PS C:\Users\sanjay\AI Assistant coding>
```

Task Description #3: Conditional Statements (Age Classification)

Scenario
You are building a basic classification system based on age.

Task
• Ask the AI tool to generate nested if-elif-else conditional statements to classify age groups (e.g., child, teenager, adult, senior).
• Analyze the generated conditions and logic.
• Ask the AI to generate the same classification using alternative conditional structures (e.g., simplified conditions or dictionary-based logic).

Expected Output #3
• A Python function that classifies age into appropriate groups.
• Clear and correct conditional logic.
• Explanation of how the conditions work.
Prompt:
Write a program that classifies a person's age into categories using conditional statements
Code:

```python
#task3
age = int(input("Enter age: "))

if age < 0:
    print("Invalid age")
elif age <= 12:
    print("Category: Child")
elif age <= 19:
    print("Category: Teenager")
elif age <= 59:
    print("Category: Adult")
else:
    print("Category: Senior Citizen")
```

Output:

Name: P.Sanjay Kumar
Roll No: 2403A52L06

```
oding/lab assignment 6.3.py"
Enter age: 20
Category: Adult
PS C:\Users\sanjay\AI Assistant coding> & C:\Use
oding/lab assignment 6.3.py"
Enter age: 5
Category: Child
PS C:\Users\sanjay\AI Assistant coding> & C:\Use
oding/lab assignment 6.3.py"
Enter age: 14
Category: Teenager
PS C:\Users\sanjay\AI Assistant coding>
```

Task Description #4: For and While Loops (Sum of First n Numbers)

Scenario
You need to calculate the sum of the first n natural numbers.

Task
• Use AI assistance to generate a sum_to_n() function using a for loop.
• Analyze the generated code.
• Ask the AI to suggest an alternative implementation using a while loop or a mathematical formula.

Expected Output #4
• Python function to compute the sum of first n numbers.
• Correct output for sample inputs.
• Explanation and comparison of different approaches.
Prompt:
Write a program that calculates the sum of first n natural numbers using both for and while loops
Code:

Name: P.Sanjay Kumar
Roll No: 2403A52L06

```python
#task4
n = int(input("Enter a natural number: "))

# Using for loop
sum_for = 0
for i in range(1, n + 1):
    sum_for += i

print("Sum using for loop:", sum_for)

# Using while loop
sum_while = 0
i = 1
while i <= n:
    sum_while += i
    i += 1

print("Sum using while loop:", sum_while)
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\sanjay\AI Assistant coding> & C:\User
oding/lab assignment 6.3.py"
Enter a natural number: 16
Sum using for loop: 136
Sum using while loop: 136
PS C:\Users\sanjay\AI Assistant coding>
```

Task Description #5: Classes (Bank Account Class)

Scenario
You are designing a basic banking application.

Task
• Use AI tools to generate a Bank Account class with methods such as deposit(), withdraw(), and check_balance().
• Analyze the AI-generated class structure and logic.
• Add meaningful comments and explain the working of the code.

Expected Output #5
• Complete Python Bank Account class.
• Demonstration of deposit and withdrawal operations with updated balance.
• Well-commented code with a clear explanation.

Name: P.Sanjay Kumar
Roll No: 2403A52L06

Prompt:
Write a python program to create a bank account class with methods to deposit , withdraw and check balance

Code:

```python
#task5
class BankAccount:
    def __init__(self, account_holder, balance=0):
        self.account_holder = account_holder
        self.balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.balance += amount
            print(f"₹{amount} deposited successfully.")
        else:
            print("Invalid deposit amount.")

    def withdraw(self, amount):
        if amount <= 0:
            print("Invalid withdrawal amount.")
        elif amount > self.balance:
            print("Insufficient balance.")
        else:
            self.balance -= amount
            print(f"₹{amount} withdrawn successfully.")

    def check_balance(self):
        print(f"Current balance: ₹{self.balance}")


# Example usage
account = BankAccount("Naveen", 1000)

account.deposit(500)
account.withdraw(300)
account.check_balance()
```

Output:

```
PS C:\Users\sanjay\AI Assistant coding> & C:\User
oding/lab assignment 6.3.py"
₹500 deposited successfully.
₹300 withdrawn successfully.
Current balance: ₹1200
PS C:\Users\sanjay\AI Assistant coding>
```

Name: P.Sanjay Kumar
Roll No: 2403A52L06

	Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.	